

Donner un agenda à un agent

Ce qui est passé reste à faire, sur le cas d'un serveur MCP Google Agenda.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.0, 2026

Sommaire

1. Le calendrier qui pardonne tout
2. Six principes, posés avant le code
3. Un seul format pour deux objets
4. Treize tools, deux familles
5. Ce que « en retard » veut dire
6. Trois préfixes dans le titre
7. Une occurrence ou toute la série
8. Ce que Google ne dit pas comme on le croit
9. La journée en un appel
10. Deux instances, un seul token
11. Les refus, avec leur message
12. Une ligne de commande avant le serveur
13. Ce que ce serveur laisse de côté
14. Ce que je retiens
15. À propos

Annexe A. Lexique

Annexe B. Revue avant de brancher un agenda

Annexe C. La configuration, clé par clé

Annexe D. La ligne de commande

1. Le calendrier qui pardonne tout

« Qu'est-ce que j'ai aujourd'hui ? »

La question est la première qu'on pose à un assistant le matin, et la réponse qu'on obtient est presque toujours fausse par omission. Un agenda sait ce qui est prévu. Il ne sait pas ce qui a eu lieu. Le rendez-vous de mardi dernier que j'ai manqué, la relance de devis notée pour jeudi et jamais faite : rien de cela ne remonte, parce qu'un calendrier traite le passage du temps comme un traitement. Une fois l'heure passée, la chose disparaît de la vue, qu'elle ait été faite ou non.

Google Tasks fait un peu mieux, puisqu'une tâche reste ouverte tant qu'on ne la coche pas. Mais il distingue « à faire » de « terminé », et rien d'autre : une tâche abandonnée et une tâche accomplie finissent dans la même colonne. Les deux produits que j'utilise tous les jours ne portent donc pas l'information qui m'intéresse le plus, celle de ce que je n'ai pas traité.

loom-plan est le serveur MCP que j'ai écrit pour que Claude Desktop réponde à la question du matin avec ce qui manque. Il lit mon agenda et mes listes de tâches, les fond dans un

seul format, et ouvre sa réponse sur les retards. Google reste l'unique référentiel : le serveur ne stocke rien en dehors du token OAuth et d'un cache de lecture de trente secondes. Le code tient en 2 400 lignes de Python réparties en douze modules, avec 726 lignes de tests pour 34 cas.

Ce cahier est une étude de cas. Il décrit une implémentation, la mienne, avec les décisions qui la structurent et les endroits où Google ne se comporte pas comme sa documentation le laisse croire. Trois choix s'en détachent et valent pour tout agenda branché sur un agent : un statut unifié qui survit au passage du temps, une date de mise en service en dessous de laquelle rien n'est en retard, et le refus de deviner quand une écriture est ambiguë.

Une chose passée n'est pas une chose faite

La règle qui commande tout le reste tient en une phrase : le passage du temps ne vaut pas traitement. Un événement dont l'heure de fin est dépassée et qui ne porte aucune marque reste ouvert, et il remonte dans la liste des retards jusqu'à ce que je le marque fait ou annulé. Il en va de même d'une tâche datée. C'est ce que le tool principal, next, place en tête de sa réponse, avant les événements du jour, avant les créneaux libres, avant les tâches.

Cette règle a une conséquence immédiate que j'ai découverte au premier appel : sans borne basse, tout l'historique de l'agenda est en retard, puisque rien de ce qui a précédé la mise en service n'a jamais été marqué. La partie 5 dit comment l'epoch règle ce point.

2. Six principes, posés avant le code

La spécification a été écrite avant la première ligne, et elle ouvre sur six principes. Ils ont tenu jusqu'à la version en service, ce qui n'était pas gagné, et chacun se retrouve quelque part dans le code décrit plus loin.

- **Google est la source de vérité.** Aucune donnée métier n'est stockée côté serveur. Un état local aurait fini par diverger.
- **Un vocabulaire pensé pour Claude.** Le modèle ne voit jamais un objet Google brut. Événements et tâches arrivent dans un seul format, en heure locale, déjà triés.
- **Lecture libre, écriture explicite.** Toute écriture n'est faite que sur ma demande. La règle figure dans la description de chacun des six tools d'écriture.
- **Rien n'est deviné.** Une ambiguïté est refusée avec un message clair. Modifier une occurrence d'une série sans dire si on vise l'occurrence ou la série est une erreur.
- **Rien ne se perd.** Ce qui est passé et non traité remonte tant qu'il n'a pas été traité.

- **Le serveur fournit les données, Claude fait la présentation.** Le format attendu, trois tableaux markdown, est prescrit dans la description du tool. La sortie est néanmoins déjà ordonnée comme le rendu voulu, pour réduire la latitude de réarrangement.

Le sixième principe mérite une remarque. J'ai préféré une sortie d'une ligne par item, avec les sections nommées, au markdown produit par le serveur : le rendu dépend de la surface, et le jour où l'hôte affichera une interface, la même sortie la nourrira sans changer le serveur. La partie 9 montre ce que le tool renvoie.

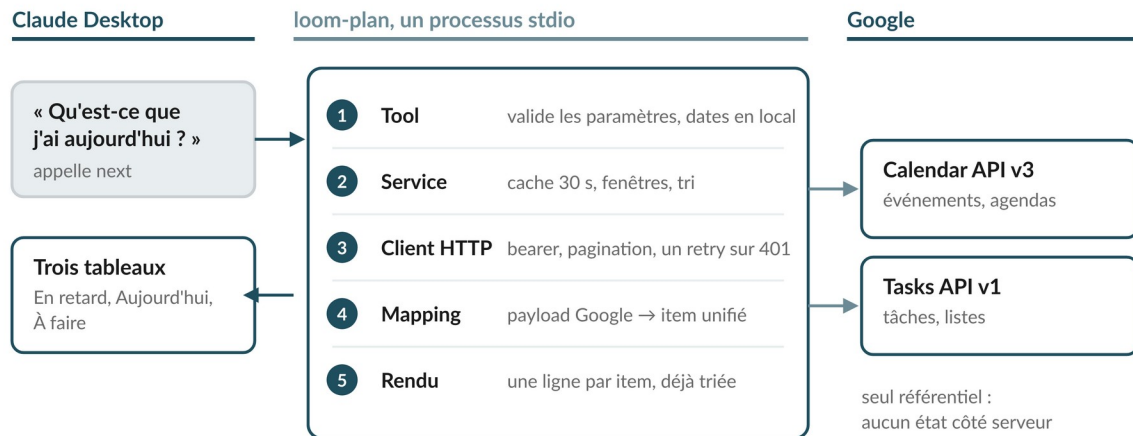
3. Un seul format pour deux objets

Un événement Google Calendar et une tâche Google Tasks n'ont rien en commun dans leur forme JSON. Le modèle unifié les ramène à seize champs, dont voici ceux qui comptent pour la suite.

Champ	Type	Ce qu'il porte
id	chaîne	evt_<agenda>/<id> ou task_<liste>/<id> : le conteneur est encodé dans l'identifiant
status	open done cancelled	le statut unifié, calculé à partir de Google et des préfixes
start, end	datetime local	événements ; pour une journée entière, end est le dernier jour inclus
due	date	tâches ; une date pure, jamais une heure
recurring, series_id, tracked	bool, chaîne, bool	occurrence d'une série, identifiant de la série, série suivie
notes	chaîne	description nettoyée de son HTML, tronquée à 500 caractères
estimate_min	entier	durée estimée, lue dans les notes sous la forme ~30min ou ~1h30

L'identifiant est le choix qui évite tout état côté serveur. Une écriture reçoit task_<liste>/<id> et sait immédiatement dans quelle liste frapper, sans table de correspondance à tenir. Le jour où j'ajouterai un second agenda, le format est déjà prêt, et aucune migration ne sera nécessaire.

Toutes les dates sortent en heure locale, Europe/Paris, au format 2026-09-14 09:30. Claude ne voit jamais d'UTC. Les entrées qu'il envoie sans fuseau sont interprétées en local, et un simple 2026-09-14 vaut minuit ce jour-là.



Le trajet d'un appel se lit sur le schéma. Claude appelle un tool, le serveur interroge l'API REST de Google avec un client HTTP asynchrone, le module de mapping convertit chaque payload en item unifié, le service trie et filtre, et le rendu produit une ligne par item. Le mapping est la partie qui a demandé le plus d'itérations, et il est couvert par dix-neuf des trente-quatre tests sur des payloads Google figés.

4. Treize tools, deux familles

Le serveur expose sept tools de lecture et six tools d'écriture. Les premiers portent l'annotation `readOnlyHint` au sens MCP, les seconds ne la portent pas, et `delete` porte en plus `destructiveHint`. Un test vérifie ces trois points sur la liste des tools telle que l'hôte la reçoit.

Tool	Ce qu'il fait	Famille
next	L'état de la journée en un appel : retards, événements, créneaux libres, tâches.	Lecture
overdue	Tout ce qui est passé et toujours ouvert, du plus ancien au plus récent.	Lecture
agenda	Événements et tâches datées sur une fenêtre de 31 jours au plus.	Lecture
tasks	Les tâches, y compris sans date. Le seul tool qui les remonte hors de next.	Lecture
find	Recherche texte sur titres et notes, 90 jours en arrière et en avant par défaut.	Lecture
free_slots	Créneaux libres calculés à partir des événements seuls, sur les jours ouvrés.	Lecture
containers	Agendas et listes disponibles, avec leur identifiant.	Lecture
add_event	Crée un événement, éventuellement récurrent et suivi.	Écriture

Tool	Ce qu'il fait	Famille
add_task	Crée une tâche. Une liste inexistante est refusée, jamais créée.	Écriture
update_event	Modifie un événement. scope obligatoire sur un récurrent.	Écriture
update_task	Modifie une tâche, y compris pour la déplacer de liste.	Écriture
set_status	open, done ou cancelled. Idempotent. Sur un récurrent, l'occurrence seule.	Écriture
delete	Suppression définitive, réservée aux erreurs de saisie.	Écriture

La consigne d'écriture, et ce qu'elle vaut

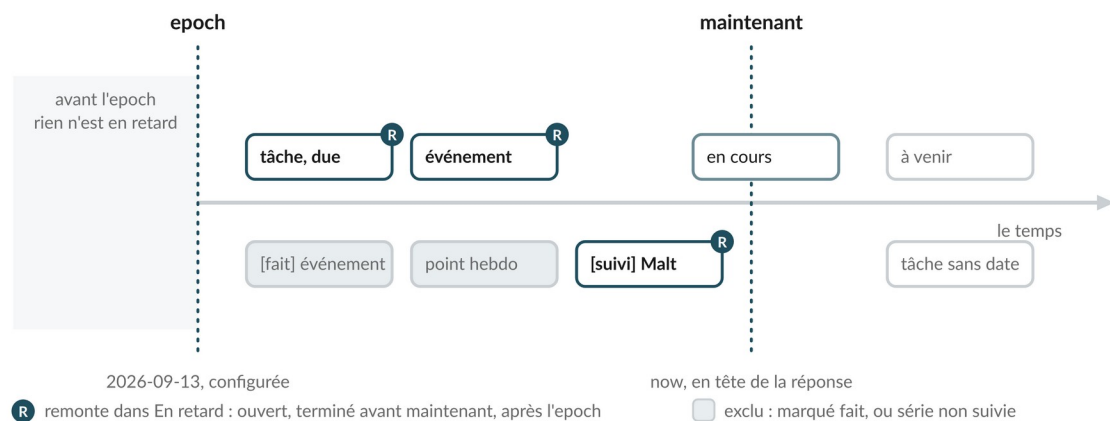
Chaque tool d'écriture commence sa description par la même phrase, et un test vérifie qu'elle y est.

ÉCRITURE : uniquement sur demande explicite de Denis. Ne jamais appeler de sa propre initiative, ni pour « aider », ni pour corriger. Renvoie l'objet résultant au format unifié.

C'est une consigne, et j'écris ici ce qu'elle vaut plutôt que de laisser croire à un verrou. Le modèle peut la suivre ou non. Ce qui la rend tenable, c'est la deuxième phrase : chaque écriture renvoie l'objet résultant, jamais un simple accusé, et une écriture que je n'ai pas demandée se voit au tour suivant. Ce compte rendu est la contre-mesure, et il ne vaut que si je le lis.

5. Ce que « en retard » veut dire

Le tool overdue applique une définition en trois clauses, et le bloc « En retard » de next est le même calcul. Est en retard toute tâche ouverte dont l'échéance est antérieure à aujourd'hui, et tout événement ouvert dont la fin est antérieure à maintenant, à condition que l'un et l'autre soient postérieurs à l'epoch. Le résultat est trié du plus ancien au plus récent.



L'epoch, ou la date à partir de laquelle on compte

L'epoch est la date de mise en service, fixée dans la configuration. Rien d'antérieur n'est considéré comme en retard. Sans elle, le premier appel aurait renvoyé des années de rendez-vous passés, puisque aucun ne portait la marque « fait ». Elle est obligatoire, et le chargement de la configuration échoue si elle manque.

Elle ne filtre que les retards. Demander l'agenda de mars reste possible, et find cherche sur ses 90 jours quelle que soit l'epoch. Un événement à cheval sur l'epoch suit la règle complète : c'est sa fin qui compte, et elle est postérieure.

Côté Calendar, l'epoch sert de borne basse à la requête : le serveur ne demande jamais à Google plus loin en arrière que la mise en service.

Les récurrences, exclues par défaut

Un point hebdomadaire passé a eu lieu. Personne ne le marque fait, et il serait absurde qu'il remonte chaque semaine en retard. Les occurrences d'événements récurrents sont donc exclues des retards, sauf demande explicite avec `include_recurring`, et sauf le cas des séries suivies, que la partie suivante décrit. La condition tient en une ligne du service.

```
for ev in await self._events_in(epoch, now, status=Status.OPEN):
    if ev.recurring and not include_recurring and not ev.tracked:
        continue
    if _event_ended(ev, now) and _event_end_exclusive(ev) > epoch:
        out.append(ev)
```

La fin exclusive mérite un mot. Pour un événement sur la journée entière, le modèle unifié porte le dernier jour inclus, et un tel événement n'est terminé qu'au lendemain de ce jour à minuit. La fonction `_event_end_exclusive` refait ce calcul au moment de comparer avec maintenant, pour qu'une journée bloquée aujourd'hui ne soit pas en retard à midi.

6. Trois préfixes dans le titre

Google Tasks distingue « à faire » de « terminé » mais pas « terminé » d'« abandonné », et Google Agenda n'a aucune notion de « fait ». Trois préfixes en tête de titre comblent ce manque. Ils sont retirés à la lecture et traduits en statut, et le serveur les réécrit lui-même à l'écriture.

Préfixe	Où	Sens
[fait]	événement	L'événement a été traité. Posé par <code>set_status(done)</code> , ou à la main depuis n'importe quelle interface Google.
[annulé]	tâche	La tâche est terminée côté Google mais c'était un abandon. Posé par <code>set_status(cancelled)</code> . On garde la trace.
[suivi]	série récurrente	Chaque occurrence est une chose à faire. Une occurrence passée sans [fait] remonte en retard. Le seul préfixe que je pose moi-même, à la création de la série.

J'ai choisi le titre plutôt que les propriétés étendues de l'API Calendar, qui auraient été invisibles. Un préfixe dans le titre se voit sur le téléphone, dans le navigateur, dans la notification, et je peux le taper depuis n'importe où. C'est un format rustique, et c'est sa qualité : il ne dépend d'aucun client.

Lire les préfixes dans n'importe quel ordre

Une occurrence suivie que je marque faite porte les deux préfixes, et leur ordre ne doit pas compter. La fonction qui analyse un titre boucle tant qu'elle trouve un préfixe en tête, sans se soucier de la casse.

```
while progress:
    progress = False
    for prefix, marked in markers:
        if prefix and rest.lower().startswith(prefix.lower()):
            rest = rest[len(prefix):].strip()
            if marked is None:
                tracked = True
            elif status is None:
                status = marked
            progress = True
```

La fonction inverse reconstruit un titre Google en plaçant d'abord le préfixe de statut, puis [suivi] si l'item est suivi, puis le titre nu. Un titre qui portait déjà des préfixes est d'abord nettoyé, ce qui évite l'accumulation quand on change deux fois de statut. Les trois chaînes sont configurables, pour qui préfère [done] ou une autre langue.

7. Une occurrence ou toute la série

Sur une récurrence, Google renvoie des occurrences dont l'identifiant est celui de la série suffixé par la date. Modifier une occurrence crée une exception de la série, et c'est

exactement le comportement voulu pour « c'est fait » : marquer faite l'occurrence de mardi ne touche pas celle de la semaine prochaine.

Pour `update_event` et `delete`, en revanche, la question se pose vraiment. Quand je demande à décaler mon point hebdomadaire, est-ce celui de cette semaine ou tous les suivants ? Le serveur refuse de choisir. Sur un événement issu d'une série, le paramètre `scope` est obligatoire, avec deux valeurs, `this` ou `series`, et son absence est une erreur dont le message explique les deux options. Sur un événement simple, le paramètre est ignoré.

Déplacer l'horaire de toute une série n'est pas pris en charge. Cela revient à réécrire l'événement maître, et les cas de bord m'ont paru trop nombreux pour une première version. Le serveur renvoie une erreur qui dit de passer par Google Agenda ou d'utiliser `scope=this`.

Créer une série suivie

`add_event` accepte une règle de récurrence au format `RRULE`, sans le préfixe, et le couple `start/end` donne la première occurrence. La règle est validée par une expression régulière avant d'être envoyée : `FREQ` obligatoire parmi quatre valeurs, puis des clauses de la forme `CLÉ=VALEUR`. Le paramètre `tracked` pose [suivi], et il n'est accepté qu'avec une récurrence, puisqu'une chose à faire une seule fois est une tâche.

```
uv run loom-plan add-event "Regarder mon compte Malt" \  
  "2026-09-16 09:00" "2026-09-16 09:30" \  
  --recurrence "FREQ=WEEKLY;BYDAY=WE" --tracked
```

8. Ce que Google ne dit pas comme on le croit

Le mapping est l'endroit où j'ai perdu le plus de temps, et chacun des points qui suivent a coûté une itération contre le vrai compte.

La fin exclusive des journées entières

Un événement sur la journée entière a chez Google une date de fin exclusive. « Du 14 au 17 » signifie du 14 au 16 inclus, et un événement d'un seul jour porte une fin au lendemain. Le serveur expose le vrai dernier jour, et refait la conversion à l'écriture en ajoutant un jour.

```
if all_day:  
  body["start"] = {"date": start.date().isoformat()}  
  # inclusive last day → exclusive end for Google  
  body["end"] = {"date": (end.date() + timedelta(days=1)).isoformat()}
```

L'échéance qui n'a pas d'heure

L'échéance d'une tâche est renvoyée sous la forme `2026-09-14T00:00:00.000Z`. Cela ressemble à un instant en UTC, et le traiter comme tel finit par décaler la date d'un jour, selon le fuseau et le sens de la conversion. L'heure n'a aucun sens : Google Tasks ne stocke pas d'heure d'échéance. Le serveur lit les dix premiers caractères et rien d'autre.

```
def parse_due(raw: str) -> date:
    return date.fromisoformat(raw[:10])
```

Des descriptions en HTML

Les descriptions d'événements arrivent en HTML dès qu'elles ont été saisies dans le navigateur. Les balises sont retirées, les retours de ligne des balises `br`, `p`, `div` et `li` sont conservés, les entités sont décodées, et le résultat est tronqué à 500 caractères avec un indicateur. Sans ce nettoyage, les notes arrivent au modèle avec leurs balises.

L'annulé qui disparaît

Un événement annulé n'est plus renvoyé par Google sans le paramètre `showDeleted`, et pour un événement simple l'annulation équivaut à une mise à la corbeille. Demander `status=cancelled` à agenda active ce paramètre. C'est la raison pour laquelle `set_status(cancelled)` sur un événement pose aussi le préfixe [annulé] dans le titre avant de changer le statut : la trace survit dans la corbeille.

9. La journée en un appel

`next` est le tool que Claude appelle pour toute question du type « qu'est-ce que j'ai aujourd'hui ». Il renvoie quatre sections dans un ordre fixe, précédées d'un en-tête qui donne l'heure courante et la fin de la fenêtre. La fenêtre va de maintenant à la fin des horaires de travail du jour, ou à minuit si on est déjà après, et le paramètre `hours` permet de la remplacer.

```
now: 2026-09-14 09:12 (lundi)
fenêtre : jusqu'à 2026-09-14 18:00

## En retard (2)
échéance 2026-09-12 — Relancer le devis Dupont — ~15 min — liste : Liste de ...
2026-09-11 14:00 → 15:00 (1 h) — Point chantier — agenda : lamard.denis@... — id : ...

## Aujourd'hui — événements (1)
2026-09-14 10:30 → 11:00 (30 min) — Appel plombier — agenda : lamard.denis@... — id : ...

## Aujourd'hui — créneaux libres (2)
libre 2026-09-14 09:12 → 10:30 (1 h 18)
libre 2026-09-14 11:00 → 18:00 (7 h)

## À faire (2)
échéance 2026-09-19 — Préparer la relance de septembre — ~1 h — liste : ... — id : ...
sans échéance — Lire la doc Tasks API — liste : Liste de lamard.denis — id : ...
```

L'ancrage temporel de la première ligne compte : un modèle ne sait pas quelle heure il est, et sans elle il ne peut pas dire ce qui est passé. La description du tool prescrit trois tableaux markdown, En retard, Aujourd'hui, À faire, avec les créneaux libres intercalés dans le second, et demande d'indiquer explicitement quand le bloc En retard est vide.

Les créneaux libres ne comptent que les événements

Une tâche ne bloque pas de temps. Elle se fait dans les heures libres à venir, et c'est justement pour la caser que je demande les créneaux. `free_slots` soustrait donc les seuls

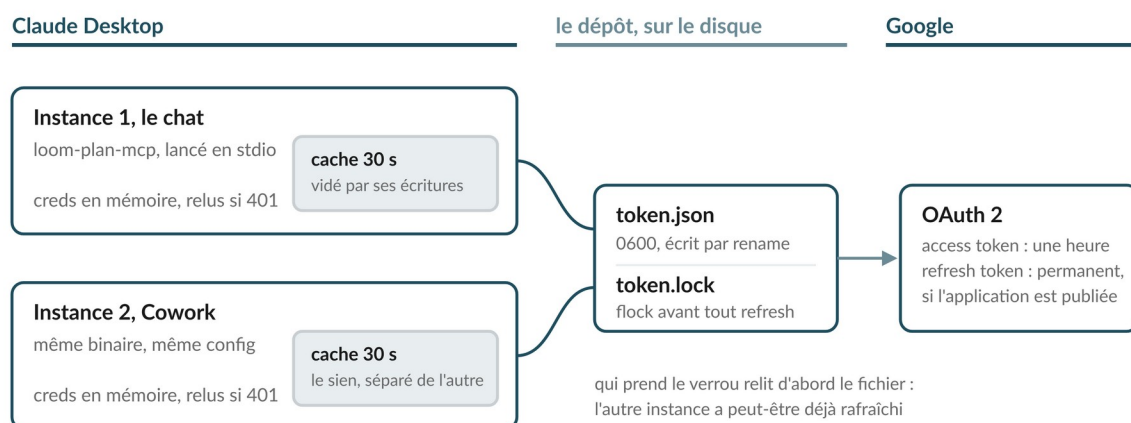
événements horaires de la fenêtre de travail, jour ouvré par jour ouvré, et rend les intervalles d'au moins trente minutes. Les journées entières ne bloquent pas. Les événements marqués [fait] bloquent toujours, puisque le temps a bien été pris, et les annulés ne bloquent plus.

Les tâches, par échéance puis par durée

Le bloc À faire est trié par échéance, les tâches sans date en dernier, puis par durée estimée croissante. La durée vient de la convention ~30min ou ~1h30 dans les notes, lue par une expression régulière. C'est un tri qui met en avant ce qu'on peut finir vite.

10. Deux instances, un seul token

Claude Desktop lance deux serveurs MCP : une instance pour le chat, une autre pour les sessions Cowork. Elles ont chacune leur cache, et rien d'autre en propre. Ce qu'elles partagent est le fichier token.json, et c'est le point que le plan de construction traite en dernier.



Un access token Google vit une heure. Quand il expire, l'instance qui s'en aperçoit le rafraîchit avec le refresh token et réécrit le fichier. Si les deux instances le font en même temps, chacune fait un aller-retour de rafraîchissement pour rien, et la dernière écriture gagne. Le rafraîchissement est donc protégé par un verrou fichier, et l'instance qui obtient le verrou relit d'abord le fichier, au cas où l'autre aurait déjà fait le travail.

```
with self._locked():
    creds = self._read() # the other instance may already have refreshed
    if not creds.valid:
        creds.refresh(Request())
    self._write(creds)
```

L'écriture du fichier est atomique, par un fichier temporaire suivi d'un rename, en permissions 0600. Côté HTTP, une réponse 401 provoque exactement une nouvelle tentative, après relecture du fichier. Un test le vérifie avec une route qui répond 401 puis 200.

Publier l'application, sinon sept jours

Le détail qui compte le plus n'est pas dans le code. Dans la console Google Cloud, un écran de consentement en statut « Test » fait expirer le refresh token au bout de sept jours, et il faut se réauthentifier chaque semaine. Passer l'application en production, sans demander de validation, suffit : l'écran « application non vérifiée » s'affiche une fois au consentement, et le token devient permanent. Les trois scopes demandés ne sont pas classés sensibles, aucune revue n'est requise.

Le flux est celui d'une application installée, lancé une fois par la ligne de commande avec `access_type=offline` et `prompt=consent`, pour garantir l'émission d'un refresh token. Si Google n'en renvoie pas, le serveur le dit et indique de révoquer l'accès dans le compte Google avant de recommencer.

11. Les refus, avec leur message

Un refus est écrit pour Claude, en clair, jamais sous la forme d'une exception Google. Un message bien formulé se corrige au tour suivant : le modèle lit les listes disponibles, repose la question, et la seconde tentative passe.

Situation	Message
liste ou agenda inconnu	la liste « X » n'existe pas ; disponibles : « Liste de lamard.denis », ...
fenêtre agenda trop large	fenêtre trop large (45 j) : agenda est limité à 31 jours, utiliser plusieurs appels
scope absent sur un récurrent	<code>update_event</code> : cet événement est une occurrence d'une série ; préciser <code>scope=this</code> (cette occurrence) ou <code>scope=series</code> (toute la série)
déplacer une série	déplacer toute une série (<code>scope=series</code>) n'est pas pris en charge en v1 ; modifier l'horaire dans Google Agenda ou utiliser <code>scope=this</code>
tracked sans récurrence	tracked ([suivi]) n'a de sens que sur une récurrence
règle RRULE invalide	règle de récurrence invalide : 'toutes les semaines' (attendu par ex. <code>FREQ=WEEKLY;BYDAY=WE</code> , <code>FREQ=MONTHLY;BYMONTHDAY=1</code> , <code>FREQ=DAILY;COUNT=10</code>)
id mal formé	id invalide : 'abc' (attendu <code>evt_...</code> ou <code>task_...</code>)
date illisible	date/heure invalide : 'lundi' (attendu « AAAA-MM-JJ HH:MM » ou « AAAA-MM-JJ »)

Deux de ces refus portent sur des choix. Une liste inexistante n'est jamais créée, parce qu'une faute d'orthographe du modèle produirait sinon une liste orpheline à chaque fois.

Et une série n'est jamais déplacée, parce que je préfère un refus net à une réécriture de récurrence que je n'ai pas testée sur tous les cas.

Les erreurs Google elles-mêmes passent par une seule fonction qui extrait le champ message du payload d'erreur et le préfixe du code HTTP. Une panne réseau devient « Google injoignable », avec le nom de l'exception. Le modèle n'a jamais à lire une trace d'appel.

12. Une ligne de commande avant le serveur

Le serveur MCP est arrivé en troisième. J'ai d'abord écrit le mapping et le service, avec des tests sur des payloads Google figés, puis une ligne de commande qui appelle le même service. C'est avec elle que le mapping s'est stabilisé contre le vrai compte, sans Claude dans la boucle : un retard réel, un marquage [fait], une suppression, et les décalages de date décrits en partie 8 sont tous apparus là.

La ligne de commande expose quatorze commandes, dont trois n'ont pas d'équivalent MCP : auth, check et serve. Les onze autres sont les tools, moins update_event et update_task. L'annexe D les donne.

Ce que les tests couvrent

Les 34 tests se répartissent en trois fichiers. Dix-neuf portent sur le mapping, avec un payload Google par cas. Treize portent sur le service, avec les API Google simulées au niveau HTTP par respx, ce qui permet de vérifier le corps exact de ce qui est envoyé et le nombre d'appels réseau, donc l'invalidation du cache par une écriture. Deux portent sur le contrat du serveur : la liste des tools, leurs annotations, la consigne d'écriture, et la ressource d'interface attachée à next.

Pyright tourne en mode strict. Les bibliothèques Google ne sont pas typées, et une façade Protocol de cinq membres isole la frontière dans le module d'authentification.

13. Ce que ce serveur laisse de côté

Cette partie existe parce qu'un lecteur qui découvre ces limites après coup a raison de se demander ce qu'on lui a caché d'autre.

L'écriture reste une consigne

Rien n'empêche techniquement le modèle d'appeler add_task de sa propre initiative. La règle est écrite dans les descriptions, l'hôte demande en général une confirmation sur les tools sans readOnlyHint, et chaque écriture renvoie ce qu'elle a fait. C'est une chaîne de trois maillons dont aucun n'est un verrou. Le protocole ne m'offre pas aujourd'hui de canal de confirmation hors de la conversation, et je vis avec.

Deux caches qui ne se parlent pas

Chaque instance a son cache de trente secondes, et une écriture n'invalide que le cache de l'instance qui l'a faite. Une tâche créée depuis le chat peut donc manquer dans une session Cowork ouverte au même moment, pendant au plus trente secondes.

Un seul agenda

Un seul agenda est lu et écrit, le mien. Le format des identifiants prépare le second sans migration, mais rien ne l'exploite encore.

Pas d'interface, pas de déclencheur

Le crochet MCP Apps est en place : next déclare une ressource HTML et une page minimale de 82 lignes attend d'être rendue. Claude Desktop lit la ressource et ne monte aucune iframe pour les serveurs personnalisés. Le déclenchement d'actions automatiques, qui était dans mon idée de départ, a été mis de côté avant la spécification.

Le serveur tourne avec vos droits

Comme tout serveur MCP en stdio, c'est un processus lancé par l'application cliente. Le token est un fichier du dépôt, ignoré par git, en 0600. Tout processus qui tourne sous mon compte peut le lire, et donc écrire dans mon agenda.

14. Ce que je retiens

La décision qui structure tout le reste est celle de l'epoch, et je ne l'avais pas prévue. Un système qui refuse d'oublier a besoin d'une date à partir de laquelle il commence à se souvenir, sinon il vous rend tout votre passé au premier appel. Une ligne de configuration, obligatoire, et le problème n'existe plus.

La deuxième est d'avoir mis les statuts dans le titre. Une convention qu'on peut appliquer à la main depuis un téléphone vaut mieux qu'un champ propre que seul le serveur sait lire. Ce que je perds en élégance, je le gagne en usage : je peux marquer une chose faite depuis le téléphone, sans passer par l'agent.

La troisième est le refus de deviner. Chaque fois que j'ai été tenté de résoudre une ambiguïté par défaut, la liste cible absente, l'occurrence ou la série, le message d'erreur a été moins coûteux à écrire que le comportement par défaut à documenter, et il se corrige au tour suivant.

Reste la question que ce cahier ne tranche pas. Un agenda lu par un agent devient un agenda que l'agent peut remplir, et la consigne d'écriture est la seule barrière. Tant que je lis ce qui revient de chaque écriture, elle tient. Le jour où cet agent tournera sans personne devant, il faudra autre chose.

15. À propos

Je conçois et j'intègre des agents IA destinés à tourner en production, pour des PME et des artisans. Ce cahier décrit un outil de mon propre poste de travail, le troisième d'une série de serveurs MCP locaux après le disque et la mémoire.

J'écris et je maintiens un socle d'orchestration public. Il porte les mécanismes décrits dans cette collection : validation des arguments avant appel, exécution parallèle bornée, checkpoints et reprise.

Le code de ce serveur est public : github.com/denislamard/loom-plan

Le socle d'orchestration : github.com/denislamard/loom

Le détail technique et les cas d'usage : denislamard.github.io

Annexe A. Lexique

Terme	Ce qu'il désigne ici
item	un événement ou une tâche, dans le format unifié que le serveur rend à Claude
conteneur	un agenda Google Calendar ou une liste Google Tasks ; encodé dans l'identifiant de chaque item
statut unifié	open, done ou cancelled, calculé à partir du statut Google et des préfixes de titre
epoch	la date de mise en service, en dessous de laquelle rien n'est en retard
occurrence	une instance d'un événement récurrent, dont l'identifiant est celui de la série suffixé par la date
série suivie	une récurrence dont le titre commence par [suivi] : chaque occurrence est une chose à faire
scope	sur un récurrent, la portée d'une écriture : this pour l'occurrence, series pour la série
consigne d'écriture	la phrase en tête de chaque tool d'écriture qui réserve l'appel à une demande explicite
refresh token	le jeton longue durée qui permet d'obtenir un access token sans consentement

Annexe B. Revue avant de brancher un agenda

Une page à parcourir avant de laisser un agent lire et écrire dans un agenda qu'on utilise vraiment, quel que soit le connecteur.

Le référentiel

- L'agenda reste la seule source de vérité, et le serveur ne garde aucune copie qui pourrait diverger.
- Un état passé et non traité remonte, on sait à partir de quelle date, et les statuts posés sont visibles dans l'interface d'origine.

L'écriture

- La liste des tools d'écriture est connue, et chacun a été appelé une fois à la main.
- Les tools de lecture portent l'annotation readOnlyHint, la suppression porte destructiveHint.
- Chaque écriture renvoie l'objet résultant, et on lit ce compte rendu.
- Une écriture ambiguë est refusée, et le message dit comment la reformuler.

Le token

- L'application est publiée dans la console Google, sinon le refresh token expire en sept jours.
- Les scopes sont les plus étroits qui permettent encore de travailler.
- Le token est dans un fichier ignoré par git, en 0600, et les instances qui le partagent se verrouillent avant de le rafraîchir.

La mesure

- Un jeu de payloads figés couvre les cas où l'API ne dit pas ce qu'on croit, et le mapping a été validé contre le vrai compte avant d'être exposé au modèle.

Annexe C. La configuration, clé par clé

Le fichier loom-plan.toml est lu depuis LOOM_PLAN_CONFIG, l'option --config ou le répertoire courant ; ses chemins relatifs sont résolus par rapport à lui. Quatorze clés.

Clé	Défaut	Rôle
[auth] client_secret	client_secret.json	le client OAuth, type Application de bureau
[auth] token	token.json	le token et le refresh token, écrits par auth
[general] timezone	Europe/Paris	fuseau des entrées et des sorties
[general] epoch	obligatoire	date de mise en service
[general] primary_calendar	primary	agenda par défaut des écritures
[general] default_task_list	obligatoire	liste par défaut d'add_task

Clé	Défaut	Rôle
[general] notes_max_chars	500	troncature des notes et des descriptions
[general] cache_ttl_seconds	30	durée du cache de lecture ; 0 le désactive
[work] start	09:00	début des horaires de travail
[work] end	18:00	fin des horaires de travail ; doit suivre start
[work] days	mon à fri	jours ouvrés, en anglais abrégé
[status] done_prefix	[fait]	préfixe d'un événement traité
[status] cancelled_prefix	[annulé]	préfixe d'une tâche abandonnée
[status] tracked_prefix	[suivi]	préfixe d'une série suivie

Annexe D. La ligne de commande

Même service et mêmes lignes que les tools ; l'option --config se place avant la commande.

Commande	Ce qu'elle fait
auth	consentement OAuth dans le navigateur, écrit token.json
check	liste agendas et listes Tasks : les trois scopes validés d'un coup
containers	agendas et listes disponibles, avec leur identifiant
serve	le serveur MCP en stdio, équivalent de loom-plan-mcp
next [--hours H]	retards, événements, créneaux, tâches
overdue [--include-recurring]	tout ce qui est passé et toujours ouvert
agenda START [END]	événements et tâches datées, 31 jours au plus
tasks [--list L] [--status S]	tâches, datées ou non
find QUERY [--from D] [--to D]	recherche texte, ±90 jours par défaut
free START END [--min M]	créneaux libres sur une fenêtre
add-task TITRE [--due D] [--notes N]	crée une tâche
add-event TITRE START END	crée un événement ; --recurrence, --tracked, --all-day
set-status ID STATUS	open, done ou cancelled
delete ID [--scope S]	suppression définitive ; préférer set-status cancelled